

Report
Of
**SMOKE AND FIRE HAZARD DETECTION SYSTEM USING
CONVOLUTIONAL NEURAL NETWORKS (CNN)**

Submitted
To
School of Computer Science and Engineering
IILM University, Greater Noida, U.P.
In partial fulfilment of the requirement of degree of
B.Tech CSE



By
Roll Number of Student: CS-2341302
Name of Student: Sneha
Batch: 2026-27

CANDIDATE'S DECLARATION

I, Sneha (Roll No. CS-2341302) do hereby declare that the internship report titled **“Smoke and Fire Hazard Detection System Using Convolutional Neural Networks (CNN)”** has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE at IILM University, Greater Noida. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature

Student Name: Sneha

Roll No.: CS-2341302

ACKNOWLEDGEMENT

I am using this opportunity to express my profound gratitude to everyone who supported me throughout the Internship Program at **BitePay**. I am thankful to my industry supervisor, Mr. Vishesh Gumber, for his inspiring guidance, invaluable constructive criticism, illuminating views on computer vision and web integration throughout this research work.

I express my sincere gratitude to the Faculty Coordinators and Department of Computer Science and Engineering, IILM University, Greater Noida, for their academic support. Finally, I express my deep gratitude to my parents for their constant encouragement and blessings.

I express my gratitude to my parents for their blessings.

Signature

Student Name: Sneha

Roll No.: CS-2341302

Chapter No.	Chapter Name	Page No.
	Cover Page	i
1.	Candidate's Declaration	ii
2.	Acknowledgment	iii
3.	Internship Completion Certificate	iv
4.	Project Description 4.1 Introduction 4.2 Organization Profile 4.3 Problem Statement 4.4 Project Objectives 4.5 Scope of the Project 4.6 Technologies and Tools Used 4.7 System Architecture 4.8 Methodology 4.9 Expected Outcomes 4.10 Certificates of completion and other communication mail proof must be attached	5 5-6 6 7 7 7-8 8 8-9 9 10
5.	Bibliography/References	11

CHAPTER 4: PROJECT DESCRIPTION

4.1 Introduction

Fire hazards pose a significant threat to life, property, and the environment. Early detection is paramount in mitigating the catastrophic consequences of fire outbreaks. Traditional fire detection systems primarily rely on smoke and heat sensors, which often require close proximity to the source and may result in delayed response times in large or open spaces.

With the rapid advancements in Artificial Intelligence (AI) and Computer Vision, deep learning algorithms, particularly Convolutional Neural Networks (CNNs), have emerged as highly effective tools for image classification and real-time hazard detection. This project presents a vision-based "Smoke and Fire Detection System" that utilizes live webcam feeds to classify scenes into three categories: Fire, Smoke, and Neutral. Upon detecting a hazard, the system immediately triggers an automated email alert to registered users, thereby ensuring rapid response and enhanced safety.

4.2 Organization Profile (BITEPAY)

This project was successfully developed during an internship at **BitePay** (<https://bitepay.in/>). BitePay is an innovative organization and a registered Udyog Aadhar MSME (Micro, Small, and Medium Enterprises) entity. The company focuses on delivering robust technological solutions, and this internship provided a dynamic environment to bridge the gap between theoretical machine learning concepts and practical, production-ready web applications.

4.3 Problem Statement

Current conventional fire alarm systems have several limitations:

1. **Response Delay:** Physical sensors (thermal or particle-based) take time to register abnormal conditions, especially in high-ceiling environments or outdoors.

2. **False Alarms:** Traditional sensors can be triggered by non-hazardous activities (e.g., cooking, dust).

3. **Lack of Remote Monitoring:** Conventional systems often lack the capability to instantly notify remote administrators or building supervisors when they are off-site.

To resolve these challenges, this project investigates a vision-based approach using a custom-trained Keras CNN model. By analyzing visual data from surveillance cameras in real-time and coupling it with an automated SMTP email alert backend, the system aims to provide an instantaneous, reliable, and remotely accessible hazard warning mechanism.

4.4 Project Objectives

The main objectives of this research internship project are:

1. **Develop a Vision Model:** Train a robust Convolutional Neural Network (CNN) using Keras/TensorFlow to accurately classify images into three distinct classes: Fire, Smoke, and Neutral.

2. **Real-time Inference Integration:** Integrate the trained AI model into a web-based frontend using JavaScript for real-time webcam frame analysis.

3. **Automated Alert System:** Develop a FastAPI backend capable of handling user subscriptions (login/logout) and dispatching instantaneous email alerts via Brevo SMTP whenever a fire or smoke hazard is detected.

4. **End-to-End Deployment:** Create a seamless hybrid client-server architecture where the frontend handles the heavy lifting of real-time computer vision inference, while the backend securely manages communications and user states.

4.5 Scope of the Project

The technical scope of the project includes:

- * **Dataset & Modeling:** Utilizing a custom dataset of Fire, Smoke, and Neutral images to train a CNN architecture on Google Colab, leveraging categorical cross-entropy loss and optimizing for high accuracy.

- * **Frontend Web Application:** A responsive dashboard built with HTML, CSS, and Vanilla JavaScript that captures live video feeds via the WebRTC API and performs client-side or server-coordinated predictions.

- * **Backend Alerting API:** A lightweight Python FastAPI server that maintains a persistent JSON database of active users (`users\_history.json`) and interfaces with the Brevo SMTP service to broadcast hazard alerts.

4.6 Technologies and Tools Used

The project implementation was realized using open-source computing, deep learning frameworks, and modern web technologies:

- * **Programming Languages:** Python 3, JavaScript (ES6+), HTML5, CSS3.

- * **Deep Learning Library:** Keras and TensorFlow (for model training in Google Colab).

- * **Backend Framework:** FastAPI with Uvicorn (ASGI server).

- * **Email Communication:** Brevo SMTP (formerly Sendinblue) for reliable transactional email delivery.

- * **Data Format & Storage:** JSON for lightweight, persistent user session management.

- * **Development Environment:** Google Colab (Model Training), VS Code / Antigravity IDE (Web Application Development).

4.7 System Architecture

The end-to-end system architecture comprises three main interconnected modules:

1. **AI Inference Module:** The core CNN model consisting of multiple Convolutional and MaxPooling layers, followed by dense layers and a softmax output to classify the presence of 'Fire', 'Smoke', or 'Neutral' states from image frames.

2. **Frontend Surveillance Dashboard:** The user interface accesses the device webcam. It continuously captures frames and evaluates the scene. It also provides a portal for users to register their email addresses to receive alerts.

3. **FastAPI Alerting Backend:** A RESTful API server featuring endpoints (`/api/login`, `/api/logout`, `/api/alert`). When the frontend detects a hazard, it POSTs a request to the backend. The backend retrieves the list of active subscribers and securely loops through the Brevo SMTP server to send warning emails instantly.

4.8 Methodology

The research execution followed a four-step experimental methodology:

- * **Phase 1 — Data Collection & Model Training:** A dataset containing varied images of fire, smoke, and normal environments was curated. The CNN model was constructed and trained in Google Colab, utilizing data augmentation techniques to prevent overfitting.

- * **Phase 2 — Backend API Development:** The FastAPI server was built to handle subscription state. Endpoints were created to append new user emails to a JSON file upon login and remove them upon logout.

- * **Phase 3 — Email Integration:** The `smtplib` library was utilized to connect to Brevo's SMTP server on port 587 (TLS). Dynamic looping was implemented to ensure all logged-in users receive notifications without failure.

- * **Phase 4 — Frontend Integration & Testing:** The webcam module was linked to the inference engine. The system was rigorously tested to ensure that the detection of Fire/Smoke immediately triggers the backend `/api/alert` route without spamming the API (using cooldown timers).

4.9 Expected Outcomes & Performance Evaluation

The integration of the CNN model with the web architecture yielded the following outcomes:

1. **High Classification Accuracy:** The custom Keras CNN demonstrated strong capability in distinguishing between the three classes, minimizing false positives (Neutral misclassified as Fire).
2. **Instantaneous Notifications:** The FastAPI and Brevo SMTP integration successfully delivered email alerts within seconds of a hazard being detected on the webcam feed.
3. **Scalable Session Management:** The JSON-based subscriber system proved effective for handling multiple simultaneous users logging in and out of the monitoring platform.
4. **Real-world Applicability:** The project successfully demonstrated a proof-of-concept for a low-cost, highly accessible fire monitoring system that can be deployed on standard hardware with an internet connection.

4.10 Certificates & Communication Proof



CHAPTER 5: BIBLIOGRAPHY / REFERENCES

- [1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," **Nature**, vol. 521, no. 7553, pp. 436-444, 2015.
- [2] K. Muhammad, J. Ahmad, I. Mehmood, S. Rho, and S. W. Baik, "Convolutional Neural Networks Based Fire Detection in Surveillance Videos," **IEEE Access**, vol. 6, pp. 18174-18183, 2018.
- [3] F. Chollet et al., **Keras**. <https://keras.io>, 2015.
- [4] S. Ramirez, "FastAPI: High performance, easy to learn, fast to code, ready for production," **FastAPI Documentation**, 2023. [Online]. Available: <https://fastapi.tiangolo.com/>
- [5] MDN Web Docs, "WebRTC API," **Mozilla**, 2023. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API